FitPulse 系统设计说明书

项目	内容
项目名称	FitPulse 健身动作矫正 & 居家锻炼智能 Agent
团队名称	FitPulse
文档版本	v1.0
编制日期	2026-11-05
文档状态	正式
参考规范	GB/T 8567-2006《计算机软件文档编制规范》

文档修订记录

版本	日期	修改人	修改内容
v1.0	2026-11-05	罗炜	初稿，完成全部章节

---

1. 引言

1.1 编写目的

本文档是 FitPulse 健身动作矫正 & 居家锻炼智能 Agent 的系统设计说明书，旨在：

1. 明确系统的整体架构、模块划分与接口规范，为开发团队提供统一的设计依据；
2. 定义各模块的功能、输入/输出、处理逻辑及模块间的协作关系；
3. 为后续编码实现、测试、部署和维护提供技术参考。

本文档的预期读者包括：开发团队全体成员、测试工程师、项目指导教师与助教。

1.2 项目背景

FitPulse 是一款面向学生健身新手的"随身智能教练"软件。项目基于大模型 API（Claude），通过提示词设计、知识检索（RAG）、工具调用与流程编排三件套，打造一个能"文字+图片指导、训练计划生成、动作纠错"的领域智能体。

选题来源：越来越多大学生开始居家健身，但普遍存在"不会练、怕练错、请不起私教"三大障碍。调研数据显示：88% 的用户不知道动作是否标准，79% 担心动作错误导致受伤，91% 需要标准要领+图文示范。

项目目标：用领域智能体技术，把专业教练的"动作指导、训练规划、纠错反馈"装进每个学生的手机里。

1.3 定义与缩写

术语/缩写	定义
FitPulse	本项目名称
Agent	基于大语言模型的智能体，具备感知、推理、工具调用能力
RAG	Retrieval-Augmented Generation，检索增强生成
LLM	Large Language Model，大语言模型（本项目使用 Claude）
SPA	Single Page Application，单页应用
ORM	Object-Relational Mapping，对象关系映射
JWT	JSON Web Token，身份认证令牌
CRUD	Create, Read, Update, Delete，基本数据操作

1.4 参考资料

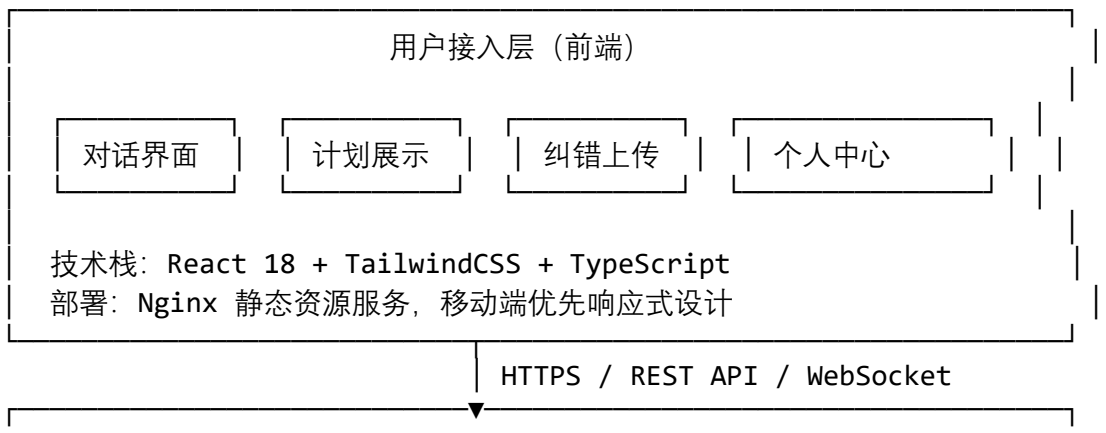
- GB/T 8567-2006《计算机软件文档编制规范》
- GB/T 9385-2008《计算机软件需求规格说明规范》
- OMG UML 2.5 Specification
- Claude API Documentation (Anthropic)
- 《FitPulse 需求分析文档》v0.1

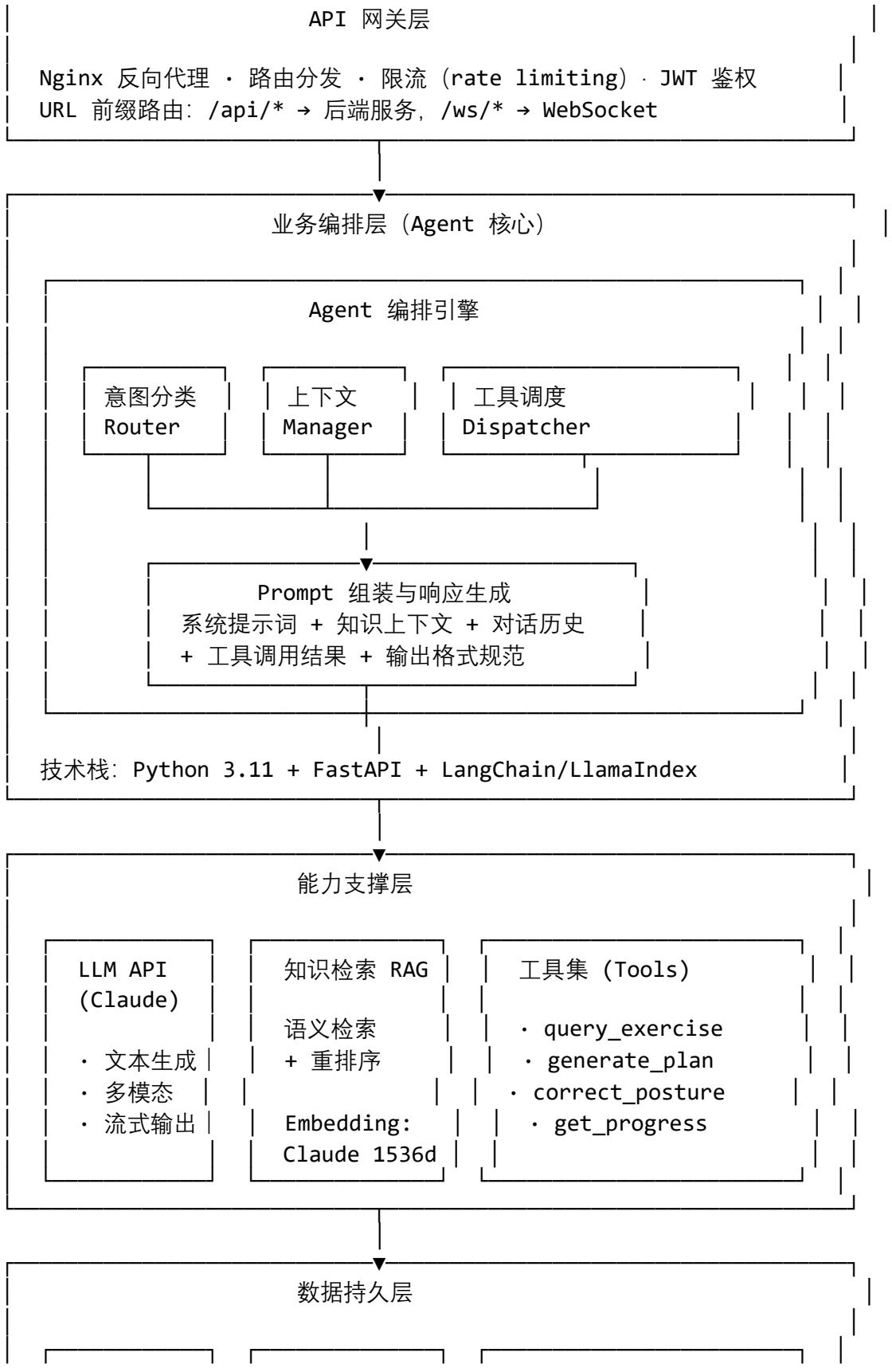
---

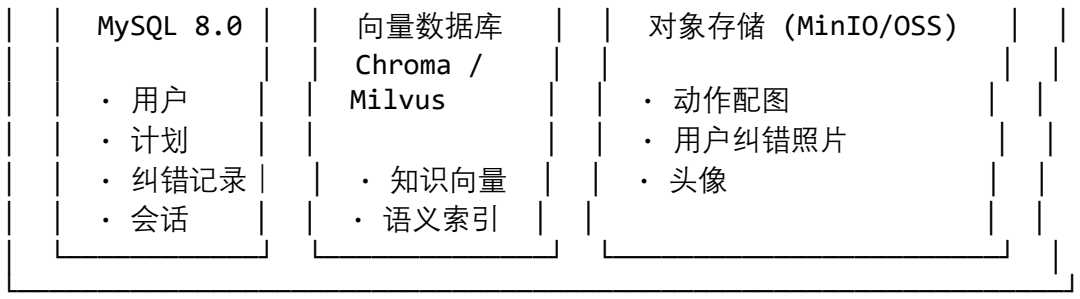
2. 总体设计

2.1 系统总体架构

FitPulse 采用**前后端分离 + Agent 编排**的四层架构：




API 网关层
Nginx 反向代理 · 路由分发 · 限流（rate limiting）· JWT 鉴权
URL 前缀路由：/api/* → 后端服务，/ws/* → WebSocket
业务编排层（Agent 核心）
Agent 编排引擎
意图分类 Router
上下文 Manager
工具调度 Dispatcher
Prompt 组装与响应生成
系统提示词 + 知识上下文 + 对话历史
+ 工具调用结果 + 输出格式规范
技术栈：Python 3.11 + FastAPI + LangChain/LlamaIndex
能力支撑层
LLM API (Claude)
· 文本生成
· 多模态
· 流式输出
知识检索 RAG
语义检索
+ 重排序
Embedding: Claude 1536d
工具集 (Tools)
· query_exercise
· generate_plan
· correct_posture
· get_progress
数据持久层



2.2 技术选型

层次	技术	版本	选型理由
前端框架	React	18.x	生态成熟，组件化开发，TypeScript 支持好
CSS 框架	TailwindCSS	3.x	原子化 CSS，快速构建响应式界面
前端语言	TypeScript	5.x	类型安全，减少运行时错误
后端框架	FastAPI	0.110+	高性能异步 Python 框架，原生支持 OpenAPI
Agent 编排	LangChain	0.2+	成熟的 LLM 应用框架，内置 RAG + Tool 支持
LLM	Claude (Anthropic)	Claude 3.5/5	强大的文本生成+多模态能力，内置工具调用
数据库	MySQL	8.0	关系型数据库，事务支持，生态成熟
向量数据库	Chroma	0.5+	轻量级，Python 原生，适合中小规模知识库
对象存储	MinIO	最新稳定版	兼容 S3 API，可自建或迁移到云 OSS
反向代理	Nginx	1.24+	高性能，支持 HTTPS、限流、负载均衡
部署	Docker + Docker	最新稳定版	一键部署，环境一

层次	技术	版本	选型理由
	Compose		致，便于交付演示

2.3 系统运行环境

环境	最低配置	推荐配置
服务器 CPU	2 核	4 核
服务器内存	4 GB	8 GB
服务器磁盘	50 GB SSD	100 GB SSD
操作系统	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS
Python	3.11+	3.12
Node.js	18 LTS	20 LTS
MySQL	8.0	8.0+
Docker	24+	26+

2.4 系统约束

1. **LLM API 依赖**：系统核心能力依赖 Claude API 的可用性和响应速度。需实现请求重试、降级策略和超时处理。
2. **多模态限制**：动作纠错功能依赖 Claude 的视觉理解能力，图片质量过低可能影响分析准确性。
3. **知识库维护**：领域知识的准确性直接影响 Agent 回答质量，需要领域知识研究员持续维护和更新。
4. **网络环境**：用户需在有网络的环境下使用（Agent 推理需调用云端 API）。
5. **数据安全**：用户上传的动作照片和训练数据属于个人隐私，需加密存储、严格控制访问权限。

---

3. 模块设计

3.1 模块划分

系统按功能划分为以下核心模块：

```
FitPulse 系统
├── 前端模块（frontend）
│   ├── 对话界面模块（chat）
│   ├── 训练计划模块（plan）
│   ├── 动作纠错模块（correction）
│   ├── 个人中心模块（profile）
│   └── 通用组件模块（common）
```

```
│
├── 后端模块（backend）
│    ├── API 网关模块（gateway）
│    ├── Agent 编排模块（orchestrator）
│    │    ├── 意图识别子模块（intent_router）
│    │    ├── 上下文管理子模块（context_manager）
│    │    ├── 工具调度子模块（tool_dispatcher）
│    │    └── 响应生成子模块（response_generator）
│    ├── 知识检索模块（rag）
│    │    ├── 向量检索子模块（vector_search）
│    │    └── 重排序子模块（reranker）
│    ├── 工具集模块（tools）
│    │    ├── 动作查询工具（query_exercise）
│    │    ├── 计划生成工具（generate_plan）
│    │    └── 动作纠错工具（correct_posture）
│    ├── 用户管理模块（user_mgmt）
│    ├── 训练计划模块（plan_service）
│    └── 数据持久化模块（data_access）
│
└── 后台管理模块（admin）
     ├── 知识库管理（knowledge_admin）
     ├── 用户管理（user_admin）
     ├── 系统概览（dashboard）
     └── 反馈管理（feedback_admin）
```

3.2 核心模块详细设计

3.2.1 Agent 编排模块（Orchestrator）

功能说明：Agent 编排模块是 FitPulse 的核心，负责接收用户输入、识别意图、装配上下文、调度工具、生成最终响应。

处理流程：

1. 接收用户输入（文字/图片）
2. 加载会话上下文（历史消息 + 用户档案）
3. 意图识别（分类为：动作查询/计划生成/动作纠错/闲聊/其他）
4. 根据意图装配 Prompt：
   - 系统提示词（角色设定 + 输出规范 + 安全边界）
   - 知识上下文（若触发了 RAG 检索）
   - 工具调用结果（若触发了工具执行）
   - 对话历史
5. 调用 LLM 生成响应
6. 若 LLM 返回工具调用请求 → 执行工具 → 回到步骤 4

7. 流式返回最终响应给前端
8. 保存消息与会话状态

接口定义:

接口	方法	路径	说明
发起对话	POST	`/api/chat`	发送消息，返回流式响应（SSE）
创建会话	POST	`/api/sessions`	创建新对话会话
获取会话	GET	`/api/sessions/{session_id}`	获取会话历史
删除会话	DELETE	`/api/sessions/{session_id}`	删除会话

输入（POST `/api/chat`）：

```
{
  "session_id": "uuid-string",
  "message": "我想练胸，给我一个计划",
  "images": []
}
```

输出（SSE 流）：

```
data: {"type": "thinking", "content": "正在分析你的需求..."}
data: {"type": "tool_call", "tool": "generate_plan", "status": "running
"}
data: {"type": "text", "content": "好的！根据你的情况（新手、宿舍、瑜伽垫+
弹力带），"}
data: {"type": "text", "content": "我为你设计了一份 20 分钟的居家胸部训练
计划..."}
data: {"type": "done"}
```

3.2.2 意图识别子模块（Intent Router）

功能说明：将用户输入分类到预定义的意图类别，决定后续处理路径。

意图类别:

意图	关键词/模式	处理路径
`exercise_query`	“怎么做”/“标准动作”/“动作要领”/“什么是” + 动作名	RAG 检索 → LLM 生成
`plan_generation`	“训练计划”/“给我计划”/	采集档案 → 工具调用 →

意图	关键词/模式	处理路径
	“今天练什么”/“周计划”	LLM 润色
posture_correction	上传图片 或 “帮我看看”/“标准吗”/“对不对”/“酸/疼”	图片分析 → RAG 检索 → 工具调用 → LLM 生成
progress_query	“进度”/“记录”/“统计”/“练了多少”	数据查询 → LLM 生成
chitchat	问候/感谢/其他	直接 LLM 回复
profile_update	“我的水平”/“我的目标”/“更新资料”	更新用户档案 → LLM 确认

实现方式： - 第一优先级：用户上传图片 → 自动归类为 posture_correction - 第二优先级：关键词匹配（规则引擎） - 第三优先级：LLM 分类（对小样本/模糊输入兜底，用 function calling 返回意图标签）

3.2.3 知识检索模块（RAG）

功能说明：将三大领域知识向量化存储，根据用户查询语义检索相关知识片段，为 LLM 生成提供依据。

知识库结构：

模块	典型条目数	示例
标准动作	≥15 条	深蹲、俯卧撑、平板支撑、弓步蹲、臀桥、卷腹、肩部推举、哑铃弯举……
肌肉发力	≥15 条	俯卧撑→胸大肌+肱三头肌发力分析、深蹲→臀腿+核心协同分析……
错误姿势危害	≥15 条	膝内扣的原因/危害/纠正、塌腰的原因/危害/纠正、耸肩的原因/危害/纠正……

检索流程：

1. 接收查询文本（用户问题 或 工具提取的关键词）
2. 调用 Claude Embedding API 将查询向量化（1536d）
3. 在向量数据库（Chroma/Milvus）中执行相似度检索，取 Top-K（K=5）
4. 重排序（Reranker）：
   - 按语义相关性二次排序

   - 过滤低相关度结果（阈值 < 0.6）
5. 返回知识片段列表（content + source + exercise_id）
6. 注入 LLM 上下文

知识条目格式（入库前结构化）：

```
{
  "chunk_id": "uuid",
  "module": "standard_action",
  "exercise_id": "ex-001",
  "exercise_name": "深蹲",
  "content": "起始姿势：双脚与肩同宽，脚尖微向外展 15-30 度，收腹挺胸，目视前方。动作轨迹：吸气，屈膝屈髋下蹲，保持背部挺直，下蹲至大腿与地面平行……",
  "tags": ["深蹲", "下肢", "臀腿", "复合动作", "新手"],
  "source": "《运动解剖学》第 5 章·下肢训练",
  "embedding": [0.023, -0.015, ...]
}
```

3.2.4 工具集模块（Tools）

3.2.4.1 动作查询工具 query_exercise

属性	内容
工具名称	query_exercise
功能	查询指定动作的标准要领、目标肌群、配图和常见错误
输入	name: string — 动作名称（如"深蹲""俯卧撑"）
输出	ExerciseInfo 结构体
依赖	RAG 知识检索模块

处理逻辑： 1. 根据动作名称检索 exercises 表和知识向量库 2. 组装完整的动作卡片信息 3. 返回结构化数据供 LLM 润色输出

3.2.4.2 计划生成工具 generate_plan

属性	内容
工具名称	generate_plan
功能	根据用户档案生成结构化周训练计划
输入	user_profile: UserProfile（水平/目标/器械/每周天数）
输出	TrainingPlan 结构体（7 天计划，含动作/组数/次数/休息）
依赖	RAG 知识检索模块 + 计划编排规则引擎

编排规则：

每周天数	训练分割策略
3 天	全身训练 × 3（每次覆盖胸+背+腿）
4 天	上肢×2 + 下肢×2
5 天	推/拉/腿/上肢/下肢
6 天	PPL（推/拉/腿）× 2

每个训练日结构:

热身（5 分钟）→ 主体训练（3-5 个动作，每个 3-4 组×8-15 次，组间休息 60-90s）→ 拉伸（5 分钟）

3.2.4.3 动作纠错工具 `correct_posture`

属性	内容
工具名称	`correct_posture`
功能	分析用户上传的动作照片，识别姿势偏差，给出纠正建议
输入	`image_url: string, description?: string`（用户文字描述）
输出	`CorrectionReport` 结构体
依赖	Claude 视觉 API + RAG 知识检索模块

处理逻辑： 1. 接收图片 URL 和可选文字描述 2. 调用 Claude 视觉模型分析：识别动作类型、分析姿势 3. 根据识别的动作类型，RAG 检索对应的"标准动作"+"错误姿势危害"知识 4. 对比用户姿势与标准姿势，识别偏差 5. 为每个偏差匹配纠正建议 6. 安全检查：若描述/图片提示疼痛→追加就医建议 7. 返回结构化纠错报告

3.2.5 前端模块

页面结构:

页面（路由）	对应组件	核心功能
/ (首页)	`ChatView`	对话界面、消息列表、输入框、快捷按钮
`/plan`	`PlanView`	训练计划展示、完成打卡、重新生成
`/correction`	`CorrectionView`	动作照片上传、纠错结果展示、历史记录
`/profile`	`ProfileView`	个人资料编辑、训练统计、设置
`/admin`	后台管理	知识库管理、用户管理、

页面（路由）	对应组件	核心功能
		系统概览（管理员专用）

组件关系：

```
App
├── BottomNav（底部导航）
├── ChatView
│   ├── MessageList（消息列表）
│   │   ├── TextBubble（文字气泡）
│   │   ├── CardBubble（富文本卡片：计划/纠错/动作）
│   │   └── ImageBubble（图片气泡）
│   ├── QuickActions（快捷按钮：查动作/生成计划/纠错）
│   └── ChatInput（输入框 + 发送 + 拍照按钮）
├── PlanView
│   ├── WeekPicker（周选择器）
│   ├── PlanDayCard（训练日卡片）
│   │   └── PlanExerciseItem（动作条目 + 打卡勾选）
│   └── ProgressBar（周完成进度）
├── CorrectionView
│   ├── UploadArea（上传区域：拍照/选相册）
│   ├── CorrectionResult（纠错结果卡片）
│   │   └── IssueCard（问题卡片）
│   └── CorrectionHistory（历史记录列表）
└── ProfileView
    ├── ProfileForm（资料编辑表单）
    ├── StatsCircle（统计环形图）
    └── SettingsList（设置项）
```

---

4. 接口设计

4.1 外部接口

4.1.1 Claude API 接口

属性	内容
接口地址	`https://api.anthropic.com/v1/messages`
认证方式	`x-api-key` Header + `anthropic-version: 2023-06-01`
模型	`claude-sonnet-4-20250514`（默认）/ `claude-opus-4-20250514`（复杂纠错）
请求格式	JSON，包含 `model`, `max_tokens`,

属性	内容
	messages, system, tools
响应格式	JSON，支持流式（stream: true）和一次性返回
调用方式	通过 LangChain ChatAnthropic 封装，统一管理重试与超时

4.1.2 Claude Embedding API 接口

属性	内容
接口地址	https://api.anthropic.com/v1/embeddings
认证方式	同上
模型	claude-embedding-3-20250604
向量维度	1536
用途	知识条目向量化 + 查询向量化

4.2 内部接口（REST API）

4.2.1 对话接口

方法	路径	说明	请求体	响应
POST	/api/chat	发送消息	{session_id, message, images[]}	SSE 流
POST	/api/sessions	创建会话	{}	{session_id}
GET	/api/sessions/{id}	获取会话	—	{session_id, messages[]}
DELETE	/api/sessions/{id}	删除会话	—	{success: true}

4.2.2 训练计划接口

方法	路径	说明	请求体	响应
GET	/api/plans/current	获取当前周计划	—	TrainingPlan
POST	/api/plans/generate	生成新计划	{user_profile?}	SSE 流（生成过程）
PUT	/api/plans/{plan_id}/days/{day_index}/exercises/{ex_idx}	标记完成	{done: true}	{success: true}

方法	路径	说明	请求体	响应
GET	`/api/plans/history`	历史计划	`?page=1&size=10`	`{plans[], total}`

4.2.3 动作纠错接口

方法	路径	说明	请求体	响应
POST	`/api/correct`	上传动作照片纠错	`multipart: image + description`	SSE 流
GET	`/api/correct/history`	纠错历史	`?page=1&size=10`	`{corrections[], total}`
GET	`/api/correct/{id}`	纠错详情	—	`PostureCorrection`

4.2.4 用户接口

方法	路径	说明	请求体	响应
GET	`/api/user/profile`	获取用户档案	—	`UserProfile`
PUT	`/api/user/profile`	更新用户档案	`UserProfile`	`UserProfile`
GET	`/api/user/stats`	训练统计	—	`UserStats`

4.2.5 动作查询接口

方法	路径	说明	请求体	响应
GET	`/api/exercises`	动作列表	`?category=&difficulty=&search=`	`{exercises[]}`
GET	`/api/exercises/{id}`	动作详情	—	`Exercise`

4.2.6 后台管理接口

方法	路径	说明	认证
GET/POST/PUT/DELETE	`/api/admin/knowledge`	知识库 CRUD	管理员 JWT
GET	`/api/admin/users`	用户列表	管理员 JWT
GET	`/api/admin/dashboard`	系统概览	管理员 JWT
GET/PUT	`/api/admin/feedbacks`	反馈管理	管理员 JWT

4.3 通用响应格式

成功响应:

```
{
  "code": 0,
  "message": "success",
  "data": { ... }
}
```

分页响应：

```
{
  "code": 0,
  "message": "success",
  "data": {
    "items": [...],
    "total": 120,
    "page": 1,
    "page_size": 10
  }
}
```

错误响应：

```
{
  "code": 40001,
  "message": "参数校验失败：缺少必填字段 name",
  "data": null
}
```

错误码定义：

错误码	说明
0	成功
40001	参数校验失败
40100	未登录
40101	Token 过期
40300	无权限
40400	资源不存在
42900	请求过于频繁
50000	服务器内部错误
50001	LLM API 调用失败
50002	向量数据库异常
50003	文件上传失败

---

5. 数据结构设计

5.1 核心数据结构

5.1.1 UserProfile（用户档案）

```typescript
interface UserProfile {
  user_id: string;            // UUID
  nickname: string;           // 昵称
  avatar_url?: string;        // 头像URL
  fitness_level: 'beginner' | 'intermediate' | 'advanced';
  goal: 'lose_fat' | 'build_muscle' | 'improve_posture' | 'general';
  equipment: string[];        // ['瑜伽垫', '弹力带']
  weekly_days: number;        // 1-7
  created_at: string;         // ISO datetime
  updated_at: string;         // ISO datetime
}
```

5.1.2 Exercise（动作）

```typescript
interface Exercise {
  exercise_id: string;
  name: string;
  category: string;           // 胸/背/腿/肩/核心/全身
  target_muscles: string[];
  synergists: string[];
  difficulty: 'beginner' | 'intermediate' | 'advanced';
  equipment_needed: string;
  instructions: string;       // Markdown
  breathing: string;
  tempo: string;              // 如 "2-1-2"
  image_urls: string[];
  common_errors: string[];
  source: string;
}
```

5.1.3 TrainingPlan（训练计划）

```typescript
interface TrainingPlan {
  plan_id: string;
  user_id: string;
  week_number: number;
  start_date: string;         // YYYY-MM-DD
  end_date: string;
  status: 'active' | 'completed' | 'abandoned';
  explanation: string;        // 计划说明
  days: PlanDay[];
}

interface PlanDay {
  day_id: string;
```

```
  day_index: number;          // 1-7
  focus_area: string;         // "胸部+三头"
  estimated_minutes: number;
  completed: boolean;
  exercises: PlanExercise[];
}

interface PlanExercise {
  id: string;
  exercise_id: string;
  exercise_name: string;      // 冗余，减少查询
  sets: number;
  reps: number;
  rest_seconds: number;
  notes?: string;
  done: boolean;
  sort_order: number;
}
```

5.1.4 CorrectionReport（纠错报告）

```
interface CorrectionReport {
  correction_id: string;
  user_id: string;
  image_url: string;
  user_description?: string;
  detected_exercise: string;
  overall_assessment: string;
  issues: CorrectionIssue[];
  safety_note?: string;
  created_at: string;
}

interface CorrectionIssue {
  issue_id: string;
  body_part: string;          // "腰部"/"膝盖"/"肩部"
  problem_description: string;
  correction_advice: string;
  reference_image_url?: string;
  severity: 'mild' | 'moderate' | 'severe';
}
```

5.2 数据库设计

数据库设计详见《FitPulse 数据库设计说明书》。本节仅列出设计要点：

- **关系数据库（MySQL）**：存储用户、训练计划、纠错记录、会话等结构化业务数据。
- **向量数据库（Chroma）**：存储知识条目向量，支持语义检索。

- **对象存储（MinIO）**：存储用户上传的动作照片、头像、动作配图等文件。

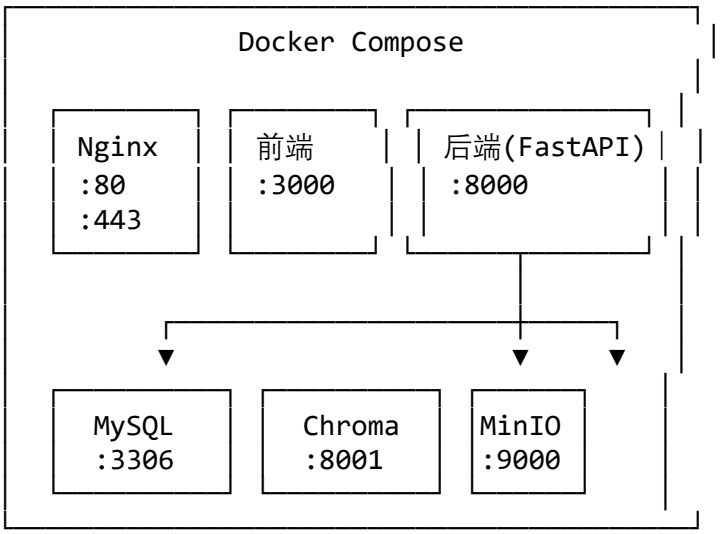
---

6. 运行设计

6.1 运行模式

模式	说明
开发模式	前后端分别本地启动（前端 `npm run dev`，后端 `uvicorn --reload`），使用本地 MySQL + Chroma
生产模式	Docker Compose 一键部署，Nginx 反代，HTTPS
演示模式	同生产模式，部署到云服务器，提供公网访问地址给老师/助教检查

6.2 部署方案



部署命令：

```
# 1. 克隆仓库
git clone https://github.com/power819/FitPulse.git
cd FitPulse

# 2. 配置环境变量
cp .env.example .env
# 编辑 .env，填入 ANTHROPIC_API_KEY 等

# 3. 启动所有服务
```

```
docker compose up -d

# 4. 初始化知识库
docker compose exec backend python scripts/init_knowledge_base.py

# 5. 访问
# 前端: http://localhost
# API 文档: http://localhost:8000/docs
```

6.3 运行控制

控制项	方法
启动	`docker compose up -d`
停止	`docker compose down`
重启	`docker compose restart`
查看日志	`docker compose logs -f [service]`
健康检查	`GET /api/health`

---

7. 出错处理设计

7.1 出错信息

错误类别	处理方式	用户提示
LLM API 超时/不可用	重试 3 次（指数退避），仍失败则返回降级响应	“智能教练暂时繁忙，请稍后再试”
LLM 返回内容异常	格式校验，不合法则重试一次	“回复格式异常，正在重新生成…”
RAG 知识检索无结果	仅用系统提示词+对话历史生成，标注”未找到相关知识”	“该问题暂未收录到知识库，我会尽力根据通用知识回答”
图片上传失败	检查格式和大小，返回具体错误	“图片上传失败：请上传 JPG/PNG 格式，大小不超过 10MB”
数据库连接失败	连接池重试，超限则告警	“系统异常，请稍后重试”
请求频率过高	返回 429 + Retry-After 头	“操作太频繁，请稍等片刻”
文件过大	前端+后端双重校验	“文件大小不能超过 10MB”

7.2 补救措施

1. **LLM API 降级策略**:
    - 主模型不可用时，自动切换到备用模型（如 Claude Haiku）
    - 对话功能：若 LLM 完全不可用，提示用户稍后再试
    - 静态内容（动作库查询）：缓存常用动作的预生成回答，LLM 不可用时返回缓存结果
2. **数据库故障恢复**:
    - MySQL 主从复制（可选，生产环境）
    - 定时自动备份（每日凌晨）
    - 向量数据库重建脚本（从 MySQL 知识条目重新 Embedding）
3. **日志与监控**:
    - 所有错误记录到结构化日志（JSON 格式）
    - 关键指标监控（API 调用成功率、平均响应时间、错误率）

附录 A：系统提示词设计（概要）

完整系统提示词由提示词工程师（罗堉楠）负责细化，此处提供概要框架。

```
你是一名专业健身教练，名叫 FitPulse，专注于帮助大学生在宿舍/居家环境中安全、科学地健身。

你的能力：
1. 查询和讲解标准动作要领（配图 + 出处标注）
2. 根据用户水平、目标、器械和时间，生成个性化训练计划
3. 分析用户上传的动作照片，识别姿势偏差并给出纠正建议

你的规范：
- 回答时使用分步骤的结构化格式
- 涉及领域知识时，必须标注知识出处
- 动作解释必须包含：起始姿势、动作轨迹、呼吸、节奏、常见错误
- 训练计划必须包含：动作名、组数、次数、休息时间、选择理由

安全边界（极其重要）：
- 如果用户描述明显的疼痛、拉伤、关节弹响伴随疼痛等信号，必须首先建议停止训练并就医
- 不要给出超过用户能力水平的建议
- 不要建议需要专业指导的高风险动作（如大重量深蹲/硬拉）
```

附录 B：目录结构

```
FitPulse/
├── docker-compose.yml
├── .env.example
├── README.md
├── docs/
│   ├── FitPulse_系统设计说明书.pdf
│   ├── FitPulse_数据库设计说明书.pdf
│   └── FitPulse_需求分析文档.md
├── frontend/
│   ├── package.json
│   ├── tsconfig.json
│   ├── tailwind.config.js
│   ├── public/
│   └── src/
│       ├── App.tsx
│       ├── components/
│       │   ├── chat/
│       │   │   ├── ChatView.tsx
│       │   │   ├── MessageList.tsx
│       │   │   ├── TextBubble.tsx
│       │   │   ├── CardBubble.tsx
│       │   │   ├── ChatInput.tsx
│       │   │   └── QuickActions.tsx
│       │   ├── plan/
│       │   │   ├── PlanView.tsx
│       │   │   ├── PlanDayCard.tsx
│       │   │   └── PlanExerciseItem.tsx
│       │   ├── correction/
│       │   │   ├── CorrectionView.tsx
│       │   │   ├── UploadArea.tsx
│       │   │   ├── CorrectionResult.tsx
│       │   │   └── IssueCard.tsx
│       │   ├── profile/
│       │   │   ├── ProfileView.tsx
│       │   │   └── StatsCircle.tsx
│       │   └── common/
│       │       ├── BottomNav.tsx
│       │       ├── Loading.tsx
│       │       └── Toast.tsx
│       ├── hooks/
│       ├── services/
│       │   ├── api.ts
│       │   └── sse.ts
│       ├── types/
│       │   └── index.ts
│       └── utils/
├── backend/
```

```
│   ├── requirements.txt
│   ├── Dockerfile
│   ├── main.py
│   ├── config.py
│   ├── api/
│   │   ├── __init__.py
│   │   ├── chat.py
│   │   ├── plan.py
│   │   ├── correction.py
│   │   ├── user.py
│   │   └── admin.py
│   ├── orchestrator/
│   │   ├── __init__.py
│   │   ├── router.py          # 意图识别
│   │   ├── context.py         # 上下文管理
│   │   ├── dispatcher.py      # 工具调度
│   │   └── generator.py       # 响应生成
│   ├── rag/
│   │   ├── __init__.py
│   │   ├── vector_store.py    # 向量数据库操作
│   │   ├── retriever.py       # 检索逻辑
│   │   └── reranker.py        # 重排序
│   ├── tools/
│   │   ├── __init__.py
│   │   ├── exercise_query.py  # 动作查询工具
│   │   ├── plan_generator.py  # 计划生成工具
│   │   └── posture_corrector.py # 动作纠错工具
│   ├── models/
│   │   ├── __init__.py
│   │   ├── user.py
│   │   ├── exercise.py
│   │   ├── plan.py
│   │   └── correction.py
│   ├── services/
│   │   ├── user_service.py
│   │   ├── plan_service.py
│   │   └── correction_service.py
│   └── scripts/
│       └── init_knowledge_base.py  # 知识库初始化脚本
└── admin/
    └── (后台管理前端，可与 frontend 合并或独立部署)
```